

UNIVERSIDAD TECNOLÓGICA DE QUERÉTARO

Unidad I - Introducción a DevOps

Proyecto Integrador

Pipeline CI/CD automatizado para API REST
Docker, GitHub Actions y AWS EC2

Dato	Información
Alumno	Axel Rodríguez
Asignatura	Introducción a DevOps
Docente	_____
Fecha	7 de octubre de 2026
Resultado local	25 pruebas aprobadas; cobertura de líneas 93.29%

Santiago de Querétaro, Querétaro

Introducción

La integración continua y el despliegue continuo permiten transformar cambios pequeños de código en versiones verificadas y disponibles de manera repetible. En un proceso manual, cada integrante debe recordar cómo ejecutar las pruebas, construir una imagen, etiquetarla, publicarla y reemplazar la aplicación en el servidor. Esta secuencia es vulnerable a omisiones, diferencias entre equipos y exposición accidental de credenciales. El propósito de este proyecto es convertir esa secuencia en un pipeline que aplique las mismas reglas a cada cambio enviado a la rama principal.

La solución parte de una API REST funcional escrita con Node.js 22 y SQLite. La aplicación administra categorías y productos mediante doce operaciones HTTP que incluyen GET, POST, PUT y DELETE. También contiene validación por JSON Schema, respuestas uniformes con las propiedades **statusCode** y **data**, autenticación administrativa para escrituras, restricciones de integridad referencial, respaldo coherente de SQLite y un endpoint de salud. Esta base permite comprobar respuestas exitosas y errores frecuentes como cuerpos incompletos, tipos incorrectos, identificadores inválidos, duplicados, relaciones inexistentes y tokens no autorizados.

La calidad se controla con el ejecutor de pruebas incluido en Node.js. La suite levanta servidores HTTP y TCP en puertos temporales, crea una base aislada y elimina sus datos al terminar. Además de las pruebas funcionales se añadieron verificaciones estáticas del workflow, de las etiquetas Docker, del uso de secretos y del orden del despliegue blue-green. El comando de cobertura impone un mínimo de 70 por ciento para líneas, funciones y ramas. Una regresión o reducción importante de cobertura impide la publicación de la imagen.

La automatización se define en GitHub Actions. En cada *push* o *pull request* hacia **main**, el job de integración descarga el código, configura Node.js y ejecuta las pruebas. Solo un push aprobado puede autenticarse en Docker Hub mediante secretos, construir la imagen y publicar las etiquetas **latest** y el hash completo del commit. La etiqueta inmutable permite identificar con precisión qué versión se despliega y facilita el diagnóstico de incidentes.

Para reducir la interrupción del servicio, AWS EC2 ejecuta dos posiciones lógicas, azul y verde. La nueva imagen se inicia en el puerto interno inactivo, se consulta repetidamente **/api/health** y solo después de una respuesta correcta Nginx cambia el tráfico público del puerto 80 mediante una recarga. El contenedor anterior se elimina después del cambio. Si la nueva versión no queda saludable, el script la retira y mantiene la versión activa. Ninguna contraseña, IP, token o clave SSH se almacena en el repositorio; la información sensible se obtiene de GitHub Secrets.

El alcance del documento incluye diseño, implementación, pruebas locales, cobertura, contenedorización, workflow y estrategia de despliegue. La publicación real en Docker Hub y el despliegue automático requieren configurar los secretos del repositorio y encender Docker Desktop. Los resultados distinguen las verificaciones completadas de los pasos externos pendientes, evitando presentar como ejecutada una evidencia que todavía no existe.

1 Arquitectura de la solución

Cambio en main	Pruebas y cobertura	Docker Hub	AWS EC2	Nginx puerto 80
git push	25 pruebas y umbral 70%	latest y SHA	contenedor azul o verde	API pública

Figura 1. Arquitectura lógica del pipeline desde el cambio de código hasta el servicio público.

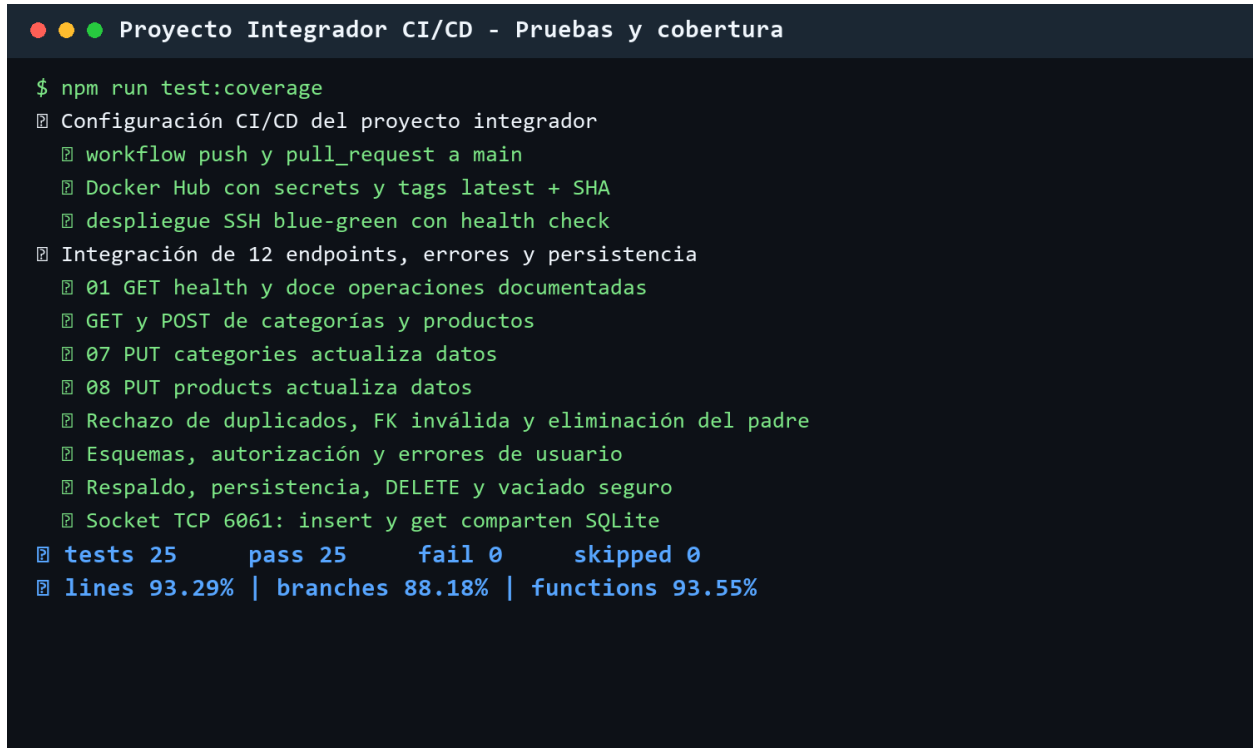
El workflow contiene tres trabajos dependientes. **test** se ejecuta para cambios y solicitudes de integración. **publish** depende de las pruebas y se limita a envíos directos a main. **deploy** consume exactamente la imagen etiquetada con github.sha, carga una clave SSH desde secretos y ejecuta el script remoto. La opción de concurrencia evita dos despliegues simultáneos.

2 Desarrollo y calidad del backend

Método	Ruta	Función
GET	/api/health	Verifica Node.js y SQLite
GET, POST	/api/categories	Lista y crea categorías
PUT, DELETE	/api/categories/{id}	Actualiza y elimina categorías
GET, POST	/api/products	Lista y crea productos
GET, PUT, DELETE	/api/products/{id}	Consulta, actualiza y elimina productos
POST	/api/database/backup	Genera respaldo consistente
DELETE	/api/database	Vacía datos y conserva estructura

3 Pruebas automatizadas y cobertura

El comando **npm run test:coverage** ejecutó 25 pruebas y todas aprobaron. Los porcentajes fueron 93.29% de líneas, 88.18% de ramas y 93.55% de funciones. Los tres indicadores superan el umbral obligatorio de 70%.



```
$ npm run test:coverage
[+] Configuración CI/CD del proyecto integrador
  [+] workflow push y pull_request a main
  [+] Docker Hub con secrets y tags latest + SHA
  [+] despliegue SSH blue-green con health check
[+] Integración de 12 endpoints, errores y persistencia
  [+] 01 GET health y doce operaciones documentadas
  [+] GET y POST de categorías y productos
  [+] 07 PUT categories actualiza datos
  [+] 08 PUT products actualiza datos
  [+] Rechazo de duplicados, FK inválida y eliminación del padre
  [+] Esquemas, autorización y errores de usuario
  [+] Respaldo, persistencia, DELETE y vaciado seguro
  [+] Socket TCP 6061: insert y get comparten SQLite
[+] tests 25      pass 25      fail 0      skipped 0
[+] lines 93.29% | branches 88.18% | functions 93.55%
```

Figura 2. Resumen verificable de pruebas y cobertura ejecutado localmente.

La suite cubre los doce endpoints, validación, autenticación, duplicados, llaves foráneas, respaldo, persistencia, vaciado y Socket TCP. También verifica que el workflow use secretos, publique dos etiquetas, no incluya direcciones IP y compruebe la salud antes de cambiar Nginx.

4 Contenedorización

El Dockerfile utiliza node:22.22.3-bookworm-slim, copia únicamente los archivos necesarios, crea el volumen /data, cambia al usuario node y define un health check. El archivo .dockerignore aplica una lista blanca: excluye todo el contexto y permite solamente el backend, los recursos públicos y el Dockerfile. Esto evita incorporar llaves, archivos .env, Git, reportes o dependencias locales.

5 Pipeline de GitHub Actions

Job	Evento	Responsabilidad
test	push y pull_request	Ejecuta pruebas y exige cobertura mínima
publish	push a main	Publica latest y el SHA en Docker Hub
deploy	push a main	Conecta por SSH y activa la versión saludable

Los secretos requeridos son DOCKERHUB_USERNAME, DOCKERHUB_TOKEN, EC2_HOST, EC2_USER, EC2_SSH_KEY, EC2_KNOWN_HOSTS y ADMIN_TOKEN. Ningún valor se encuentra en el código.

6 Despliegue blue-green

El script selecciona el color inactivo, descarga la imagen inmutable y monta el volumen persistente de SQLite. Después de confirmar la salud, genera una configuración nueva de Nginx, valida su sintaxis y recarga el servicio. Finalmente registra el color activo y elimina la versión anterior. Si el health check falla, conserva la versión anterior.

7 Resultados

Criterio	Resultado	Estado
API REST	12 operaciones	Cumplido
Pruebas automatizadas	25 aprobadas, 0 fallidas	Cumplido
Cobertura de líneas	93.29%	Cumplido
Cobertura de ramas	88.18%	Cumplido
Cobertura de funciones	93.55%	Cumplido
Dockerfile y dockerignore	Revisión estática aprobada	Cumplido
Workflow main.yml	Creado y validado por pruebas	Cumplido
Build Docker local	Motor Docker apagado	Pendiente
Push y despliegue reales	Secretos no configurados	Pendiente

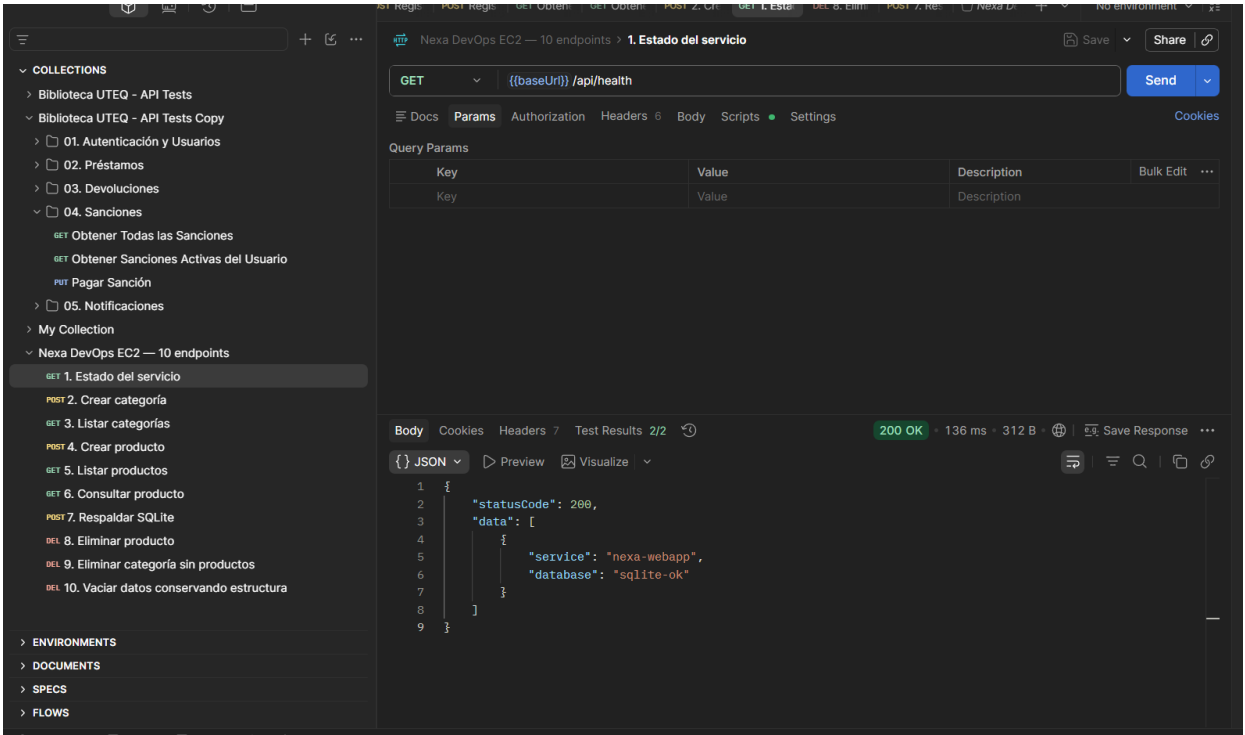


Figura 3. Respuesta HTTP 200 del endpoint de salud desplegado previamente en AWS EC2.

8 Procedimiento de demostración

Para la demostración se modifica una línea visible en la respuesta de salud, se ejecuta localmente `npm run test:coverage` y se crea un commit limitado al proyecto. El push a main debe mostrar en GitHub Actions los jobs test, publish y deploy. Después se consulta la URL pública por el puerto 80 para confirmar que la versión corresponde al hash del commit.

Las evidencias que deben añadirse después del primer pipeline real son: resumen verde de GitHub Actions, repositorio de Docker Hub con latest y SHA, salida del health check de despliegue y URL pública actualizada. No deben aparecer tokens, direcciones privadas ni contenido de la llave PEM.

9 Conclusiones

El proyecto convirtió una API funcional en un artefacto verificable y preparado para entrega continua. La cobertura quedó ampliamente por encima del mínimo: 93.29% de líneas, 88.18% de ramas y 93.55% de funciones. Las pruebas también verificaron validación, autenticación, integridad de SQLite, respaldo, persistencia y estructura de CI/CD. Esta combinación reduce la probabilidad de publicar una versión que falle por errores conocidos.

La estrategia blue-green separa la construcción de la activación. La imagen nueva debe responder correctamente antes de recibir tráfico y Nginx cambia de destino mediante una recarga. La seguridad se mantiene al usar GitHub Secrets, una etiqueta inmutable por commit, validación de `known_hosts`, ejecución sin privilegios y un archivo de entorno protegido en EC2. Quedan como pasos operativos encender Docker Desktop y registrar los secretos; una vez realizados, un git push a main demostrará el ciclo completo solicitado.

10 Fuentes de información

1. GitHub. *Understanding GitHub Actions*. <https://docs.github.com/actions>
2. Docker. *Build and push Docker images*. <https://docs.docker.com/build/ci/github-actions/>
3. Docker. *Personal access tokens*. <https://docs.docker.com/security/access-tokens/>
4. Amazon Web Services. *Get started with Amazon EC2*.
https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/EC2_GetStarted.html
5. Node.js. *Test runner and code coverage*. <https://nodejs.org/api/test.html>